Job Application Autopilot

Development Plan

From Project Setup to Deployment-Ready MVP

Project	Job Application Autopilot
Stack	FastAPI, React, PostgreSQL, LangChain, Claude API, python-docx
Milestones	8 milestones, ready for deployment
Goal	Portfolio-quality, deployment-ready resume tailoring tool

Project Overview

Job Application Autopilot is a resume tailoring tool that accepts an uploaded DOCX resume and a pasted job description, then produces a tailored resume that best represents the applicant without fabricating experience. It is built as a portfolio project to demonstrate AI workflow design skills.

The core of the product is a six-step sequential LLM pipeline:

- Parse resume content
- Analyze job requirements
- Match and map qualifications
- Tailor resume content
- Verify and validate output (hallucination safeguard)
- Generate tailored DOCX

This plan organizes development into 8 milestones. Each milestone ends with a QC checklist that must be completed before moving to the next milestone. Stretch goals are called out where relevant and are optional for the core MVP.

MILESTONE 1: Project Setup and Environment

Establish the foundation before writing a single line of application code

1.1 Repository and Version Control

- ☐ Initialize a Git repository with a clear .gitignore for Python, Node, and environment files
- ☐ Create a top-level monorepo structure: /backend and /frontend directories
- ☐ Set up a main branch and establish a feature-branch workflow (e.g., feature/*, fix/*)
- ☐ Write an initial README.md with project overview and local setup instructions
- ☐ Create a CHANGELOG.md to track version history

1.2 Backend Environment (FastAPI)

- ☐ Create a Python virtual environment inside /backend
- ☐ Set up requirements.txt (or pyproject.toml) with initial dependencies: fastapi, uvicorn, sqlalchemy, alembic, psycopg2-binary, python-dotenv, langchain, anthropic, python-docx
- ☐ Create /backend/app directory with initial __init__.py
- ☐ Add a basic FastAPI app entry point (main.py) with a health check endpoint GET /health
- ☐ Confirm the server starts locally with uvicorn and the health check returns 200

1.3 Frontend Environment (React)

- ☐ Scaffold a new React app inside /frontend using Create React App or Vite
- ☐ Remove default boilerplate and set up a clean project structure: /components, /pages, /services, /hooks
- ☐ Install initial dependencies: axios (HTTP client), react-router-dom
- ☐ Confirm the React dev server starts and renders a placeholder page

1.4 Database Setup

- ☐ Provision a PostgreSQL instance on DigitalOcean
- ☐ Configure firewall rules to allow connections only from your dev IP and future app server IP
- ☐ Create the application database and a dedicated app user with limited privileges
- ☐ Store the DATABASE_URL in a .env file (never committed to Git)
- ☐ Initialize Alembic inside /backend: alembic init alembic
- ☐ Confirm Alembic can connect to the database with alembic current

1.5 Environment Configuration

- ☐ Create a .env.example file listing all required environment variables without values
- ☐ Add ANTHROPIC_API_KEY, DATABASE_URL, and ENV (development/production) as required vars
- ☐ Confirm all secrets load correctly via python-dotenv in the FastAPI app

Milestone 1 QC Checklist

- □ Git repository initialized with .gitignore; no secrets committed
- □ Backend server starts and GET /health returns HTTP 200
- □ Frontend dev server starts and renders without errors
- □ Alembic connects to DigitalOcean PostgreSQL with alembic current
- □ .env.example is complete; actual .env is gitignored
- □ README documents how to run backend and frontend locally

MILESTONE 2: Database Schema and Data Layer

Define and migrate all tables; build the SQLAlchemy models and data access layer

2.1 Schema Design and Migration

- ☐ Define Users table: id (UUID), created_at, updated_at
- ☐ Define UserProfiles table: id, user_id (FK), raw_input (text), classified_data (JSONB), created_at, updated_at
- ☐ Define TailoringJobs table: id (UUID), user_id (FK), status (enum: pending/processing/complete/failed), job_description (text), original_resume_path (text), tailored_resume_path (text), pipeline_log (JSONB), created_at, updated_at
- ☐ Write Alembic migration for all three tables
- ☐ Run alembic upgrade head and confirm tables exist in the database

2.2 SQLAlchemy Models

- ☐ Create /backend/app/models/ directory
- ☐ Write User, UserProfile, and TailoringJob ORM models in separate files
- ☐ Add __repr__ methods to each model for clean debugging output
- ☐ Create a database session factory in /backend/app/db.py using SQLAlchemy
- ☐ Write a get_db dependency function for FastAPI dependency injection

2.3 Schema Validation (Pydantic)

- ☐ Create /backend/app/schemas/ directory
- ☐ Write Pydantic request/response schemas for UserProfile (create, read)
- ☐ Write Pydantic schemas for TailoringJob (create, read, status update)
- ☐ Ensure JSONB fields serialize cleanly to/from Python dicts

Milestone 2 QC Checklist

- ☐ alembic upgrade head runs without errors from a clean database
- ☐ All three tables exist with correct columns and types via manual DB inspection
- ☐ SQLAlchemy session can write a test record and read it back in a Python script
- ☐ Pydantic schemas validate correctly; invalid data raises clear validation errors
- ☐ No raw SQL strings in model or schema files

MILESTONE 3: User Profile API

Build the endpoint that accepts freeform user background and classifies it via LLM

3.1 Profile Input Endpoint

- ☐ Create POST /api/profiles endpoint that accepts a freeform text body
- ☐ Pass freeform text to an LLM classification call via LangChain
- ☐ LLM prompt instructs the model to classify content into: job_experience, education, projects, other_experience, skills
- ☐ Return structured JSON from the LLM and validate it against the UserProfile Pydantic schema
- ☐ Persist the raw input and classified JSONB data to the UserProfiles table
- ☐ Return the created profile (id + classified data) in the response

3.2 Profile Read Endpoint

- ☐ Create GET /api/profiles/{profile_id} endpoint
- ☐ Return the stored profile including classified_data JSONB
- ☐ Return 404 with a clear error message if profile not found

3.3 LLM Classification Prompt

- ☐ Write a dedicated system prompt for the classification LLM call
- ☐ Prompt specifies exact output JSON structure with all five category keys
- ☐ Prompt instructs the model to leave categories empty (not omit them) when no content applies
- ☐ Test the prompt manually with 3+ varied freeform inputs before wiring to the endpoint

3.4 Error Handling

- ☐ Handle LLM API errors gracefully (timeout, rate limit, malformed response)
- ☐ If LLM returns invalid JSON, log the raw response and return HTTP 502 with an informative message
- ☐ Wrap all database writes in try/except with rollback on failure

Milestone 3 QC Checklist

- ☐ POST /api/profiles with varied freeform inputs returns correctly classified JSON
- ☐ Classified data is persisted to the database and retrievable via GET /api/profiles/{id}
- ☐ Sending clearly malformed or empty input returns a useful error, not a 500
- ☐ LLM classification tested with at least 5 distinct inputs covering all five categories
- ☐ No API keys logged or exposed in any response or log output

MILESTONE 4: Resume Upload and Parsing

Accept DOCX uploads, extract structured content, store it for the pipeline

4.1 File Upload Endpoint

- ☐ Create POST /api/jobs endpoint accepting a DOCX file upload and a job_description text field
- ☐ Validate uploaded file type (must be .docx); reject other formats with HTTP 422
- ☐ Validate file size; reject uploads above a reasonable limit (e.g., 5MB)
- ☐ Save uploaded DOCX to a local /uploads directory with a UUID filename
- ☐ Store the file path in a new TailoringJob record with status: pending
- ☐ Return the job_id in the response so the client can poll for status

4.2 DOCX Parsing

- ☐ Create a parse_resume utility using python-docx
- ☐ Extract paragraphs, headings, and table content into a structured dict
- ☐ Preserve section structure (Summary, Experience, Education, Skills) where detectable
- ☐ Handle documents with no clear section headings gracefully (fall back to flat paragraph list)
- ☐ Write unit tests for the parser with at least 3 different resume formats

4.3 Job Status Endpoint

- ☐ Create GET /api/jobs/{job_id} endpoint
- ☐ Return current job status, created_at, and tailored_resume_path when complete
- ☐ Return 404 for unknown job IDs

Milestone 4 QC Checklist

- ☐ Uploading a valid DOCX returns a job_id and creates a DB record with status: pending
- ☐ Uploading a non-DOCX file (e.g., .pdf, .txt) returns HTTP 422
- ☐ DOCX parser correctly extracts content from at least 3 structurally different test resumes
- ☐ Parser handles empty sections, tables, and bulleted lists without crashing
- ☐ GET /api/jobs/{id} returns correct status for pending and nonexistent jobs

MILESTONE 5: The Six-Step LLM Pipeline

Implement the core tailoring pipeline with hallucination safeguards

5.1 Pipeline Architecture

- ☐ Create /backend/app/pipeline/ directory with a dedicated module for each step
- ☐ Create a pipeline orchestrator function that runs all six steps in sequence
- ☐ After each step, log the output to pipeline_log JSONB in the TailoringJob record
- ☐ Update job status to processing when pipeline starts and complete or failed on finish
- ☐ Wrap the full pipeline in a try/except; on any failure set status: failed and log the error

5.2 Step 1: Parse Resume

- ☐ Convert python-docx extracted content into a clean normalized dict for the pipeline
- ☐ Confirm all key sections are represented before passing to Step 2

5.3 Step 2: Analyze Job Requirements

- ☐ Write an LLM prompt to extract required skills, preferred qualifications, responsibilities, and keywords from the job description
- ☐ Return structured JSON with keys: required_skills, preferred_skills, responsibilities, keywords
- ☐ Test this prompt against at least 5 real job descriptions from different industries

5.4 Step 3: Match and Map

- ☐ Write an LLM prompt that takes the parsed resume and the job requirements analysis
- ☐ Identify which resume items are directly relevant, which can be reframed, and which are not relevant
- ☐ Return a mapping dict that the tailoring step will use to make decisions
- ☐ Prompt explicitly instructs the model not to invent experience that is not in the source resume

5.5 Step 4: Tailor Content

- ☐ Write an LLM prompt that uses the match mapping to rewrite resume sections
- ☐ Rewrite bullet points to emphasize relevance to the job description
- ☐ Adjust the professional summary to align with the target role
- ☐ Prompt instructs the model to only use facts present in the original resume
- ☐ Return a fully rewritten resume dict in the same structure as the parsed input

5.6 Step 5: Verify and Validate (Hallucination Safeguard)

- ☐ Write a dedicated verification LLM prompt that compares the tailored resume against the original

- ☐ Prompt checks for: invented companies, fabricated dates, inflated titles, skills not in the original, or quantified achievements not present in the source
- ☐ If issues are found, the verifier returns a list of specific problems
- ☐ If the verifier flags any issues, the orchestrator either requests a correction pass or fails the job with a logged explanation
- ☐ Under no circumstances does a resume with flagged hallucinations proceed to DOCX generation

5.7 Step 6: Generate DOCX

- ☐ Use python-docx to generate a clean, formatted DOCX from the verified tailored resume dict
- ☐ Apply consistent formatting: fonts, heading sizes, bullet style
- ☐ Save the output DOCX to /outputs with a UUID filename
- ☐ Update the TailoringJob record with the output path and status: complete

5.8 Pipeline Trigger

- ☐ Trigger the pipeline immediately after job creation in Milestone 4's POST /api/jobs endpoint
- ☐ For MVP, run the pipeline synchronously (consider async/background tasks as a stretch goal)

Milestone 5 QC Checklist

- ☐ Full pipeline runs end-to-end from a DOCX upload and produces a tailored DOCX output
- ☐ All six steps log their output to pipeline_log in the database
- ☐ Job status transitions correctly: pending > processing > complete (or failed)
- ☐ Verification step successfully flags a test resume that has been manually injected with a fabricated job title
- ☐ Verification step passes a clean resume with no fabrications
- ☐ Generated DOCX opens without errors in Microsoft Word and Google Docs
- ☐ Pipeline handles an LLM timeout without crashing the server
- ☐ All five prompts tested individually with edge-case inputs before pipeline integration

MILESTONE 6: React Frontend

Build the user-facing interface for profile input, resume upload, and output download

6.1 App Layout and Routing

- ☐ Set up react-router-dom with routes: / (home), /profile (profile setup), /upload (job upload), /status/:jobId (job status)
- ☐ Create a persistent top navigation bar with links to each section
- ☐ Create a shared Layout component wrapping all pages

6.2 Profile Setup Page

- ☐ Create a large freeform textarea for entering background information
- ☐ Add placeholder text explaining what kinds of information to include
- ☐ On submit, POST to /api/profiles and display the classified result as a readable summary
- ☐ Store the returned profile_id in local state or localStorage for use in the upload step
- ☐ Show a loading state while the API call is in progress
- ☐ Show a clear error message if the API call fails

6.3 Job Upload Page

- ☐ Create a file input that accepts only .docx files
- ☐ Create a large textarea for pasting the job description
- ☐ On submit, POST to /api/jobs with the file and job description
- ☐ On success, redirect to /status/:jobId
- ☐ Validate on the client that a file is selected and the job description is not empty before submitting
- ☐ Show upload progress or a spinner during submission

6.4 Job Status Page

- ☐ Poll GET /api/jobs/:jobId every 3 seconds while status is pending or processing
- ☐ Display current status with a visual indicator (spinner, progress bar, or step display)
- ☐ When status is complete, display a download button for the tailored DOCX
- ☐ When status is failed, display a clear error message and a retry or start-over option
- ☐ Stop polling once a terminal status (complete or failed) is reached

6.5 API Service Layer

- ☐ Create /frontend/src/services/api.js with typed functions for all API calls
- ☐ Centralize the base URL so it can be toggled between dev and production via environment variables
- ☐ Handle non-2xx responses uniformly and surface them to the component layer

6.6 Styling and UX

- ☐ Apply consistent, clean styling across all pages (use a CSS framework or custom CSS)
- ☐ All interactive elements (buttons, inputs) have visible hover and focus states
- ☐ The app is usable on mobile viewports (basic responsive layout)
- ☐ No unstyled HTML elements visible in the final UI

Milestone 6 QC Checklist

- ☐ Full user flow works end-to-end: enter profile > upload resume + job description > download tailored DOCX
- ☐ Profile classification result displays clearly after submission
- ☐ Status page polls correctly and stops when job reaches a terminal state
- ☐ Client-side validation prevents submitting the upload form without a file or job description
- ☐ Error states are displayed clearly for all three pages
- ☐ App renders correctly on Chrome, Firefox, and Safari
- ☐ App is usable at 375px (mobile) and 1280px (desktop) viewport widths

MILESTONE 7: Testing and Quality Assurance

Systematic testing before deployment to build confidence in every layer

7.1 Backend Unit Tests

- ☐ Set up pytest in /backend with a conftest.py that creates a test database
- ☐ Write unit tests for the DOCX parser covering: standard resume, no headings, table-based layout, empty sections
- ☐ Write unit tests for all five LLM prompt functions using mocked LangChain responses
- ☐ Write unit tests for the verification step: one passing case, one case with a flagged hallucination
- ☐ Write unit tests for all Pydantic schemas: valid data, missing fields, wrong types
- ☐ Achieve at least 80% coverage on /backend/app/pipeline/

7.2 API Integration Tests

- ☐ Write integration tests for POST /api/profiles with real DB writes (test DB)
- ☐ Write integration tests for POST /api/jobs: valid DOCX, invalid file type, missing job description
- ☐ Write integration tests for GET /api/jobs/{id}: existing job, nonexistent job
- ☐ Test that job status transitions correctly through the pipeline (mock the LLM calls)

7.3 End-to-End Pipeline QA

- ☐ Run the full pipeline with at least 5 real resumes and 5 real job descriptions
- ☐ For each run, manually review the tailored DOCX output for quality and accuracy
- ☐ Confirm no fabricated information appears in any output
- ☐ Test a resume that has very little overlap with the job description (edge case)
- ☐ Test a resume that has very high overlap with the job description
- ☐ Test with a very long job description (500+ words) and a very short one (under 50 words)

7.4 Frontend QA

- ☐ Manually walk through the full user flow in Chrome, Firefox, and Safari
- ☐ Test the upload flow with: a valid DOCX, a PDF (should be rejected), a file over the size limit
- ☐ Test the status page by refreshing mid-poll to confirm state is preserved
- ☐ Test what happens if the backend is unreachable (network error simulation)
- ☐ Verify all form validations trigger correctly before submission

7.5 Security and Hygiene

- ☐ Confirm no API keys, database credentials, or secrets appear in any API response or frontend bundle
- ☐ Add CORS configuration to FastAPI, restricting origins to the frontend domain
- ☐ Confirm uploaded files are stored with UUID names (no user-controlled filenames on disk)

- ☐ Confirm the app does not expose internal error stack traces to API consumers
- ☐ Review .gitignore to confirm .env, /uploads, and /outputs are excluded

Milestone 7 QC Checklist

- ☐ pytest suite runs cleanly with no failures
- ☐ Code coverage for /pipeline/ is at or above 80%
- ☐ All 5 real end-to-end runs produced accurate, non-hallucinated DOCX output
- ☐ No fabricated content detected in any of the 5 test runs
- ☐ CORS is configured and tested
- ☐ No secrets visible in frontend JS bundle or API responses
- ☐ All file upload edge cases (wrong type, oversized) handled gracefully

MILESTONE 8: Deployment and Launch Readiness

Get the app running on a real server and confirm it is production-ready

8.1 Server Provisioning (DigitalOcean)

- ☐ Create a Droplet (Ubuntu, minimum 2GB RAM recommended for the FastAPI process)
- ☐ Set up SSH access with key-based auth; disable password login
- ☐ Configure UFW firewall: allow ports 22 (SSH), 80 (HTTP), 443 (HTTPS), block all others
- ☐ Install Python, pip, Node.js, npm, and Nginx on the Droplet
- ☐ Create a dedicated non-root user to run the application

8.2 Backend Deployment

- ☐ Clone the repository onto the Droplet
- ☐ Set up the Python virtual environment and install requirements
- ☐ Create a .env file on the server with production values (DATABASE_URL, ANTHROPIC_API_KEY, ENV=production)
- ☐ Run alembic upgrade head against the production database
- ☐ Set up a systemd service to run uvicorn and restart it automatically on failure
- ☐ Confirm the FastAPI app starts and GET /health returns 200 on the server

8.3 Frontend Build and Deployment

- ☐ Set the production API base URL in the React environment config
- ☐ Run npm run build to produce a static build in /frontend/dist or /build
- ☐ Copy the frontend build to the Droplet (e.g., /var/www/autopilot)

8.4 Nginx Configuration

- ☐ Configure Nginx to serve the React static build on port 80
- ☐ Configure Nginx as a reverse proxy for /api/* routes to the FastAPI service on localhost
- ☐ Configure Nginx to serve /outputs/* for DOCX file downloads
- ☐ Test that the full app is accessible and functional over HTTP

8.5 HTTPS (TLS)

- ☐ Point a domain name (or subdomain) at the Droplet IP
- ☐ Install Certbot and obtain a Let's Encrypt TLS certificate
- ☐ Configure Nginx to redirect all HTTP traffic to HTTPS
- ☐ Confirm the certificate auto-renewal timer is active

8.6 Final Production Smoke Test

- ☐ Visit the live URL in a browser and confirm the frontend loads over HTTPS
- ☐ Complete the full user flow end-to-end on production: profile entry > upload > download
- ☐ Confirm the downloaded DOCX opens correctly in Word and Google Docs
- ☐ Check server logs for any errors during the smoke test run
- ☐ Confirm the database has the expected records after the smoke test

8.7 Portfolio Presentation

- ☐ Update README with: project description, tech stack, architecture summary, local setup steps, live demo URL
- ☐ Add a brief description of the six-step pipeline and the hallucination safeguard
- ☐ Add screenshots or a short screen recording of the app in action
- ☐ Optionally, link to or include the architecture documents in the repository

Milestone 8 QC Checklist

- ☐ App is live and accessible at an HTTPS URL
- ☐ Full end-to-end flow works on production without errors
- ☐ systemd service restarts the backend automatically after a manual kill
- ☐ HTTP redirects to HTTPS for all routes
- ☐ No .env files, secrets, or debug output accessible from the browser
- ☐ README is complete and clearly explains the project to a recruiter
- ☐ Repository is clean: no temporary files, test artifacts, or large binaries committed

Stretch Goals (Post-MVP)

The following are optional enhancements to pursue after the core MVP is complete and deployed. They are not required for the portfolio deliverable but would strengthen the project further.

Resume Template Library

- Design 2-3 distinct DOCX resume templates (modern, traditional, minimalist)
- Allow users to select a template before generating the tailored output
- Update the DOCX generation step to apply the selected template's formatting

Async Pipeline Processing

- Move the LLM pipeline to a background task using FastAPI BackgroundTasks or Celery
- Frontend polls for status without blocking the upload response
- This also enables better error recovery and retry logic

User Accounts and History

- Add authentication (e.g., email/password or OAuth via a provider)
- Allow users to save their profile and reuse it across multiple job applications
- Store tailoring job history so users can revisit past outputs

Cover Letter Generation

- Add a seventh pipeline step that generates a tailored cover letter from the same inputs
- Return both the resume and cover letter as a ZIP download

Pipeline Quality Scoring

- After verification, return a confidence score for the tailoring quality
- Display this score to the user as an indicator of how well the resume matches the role